

✉ BALLOO PLATFORM — Интерактивные Функции Сообщений V1

Версия: v5.0

Дата: 2026-06-29

Статус: ☒ Полная спецификация V1

ОГЛАВЛЕНИЕ

1. Обзор
 2. Пересылка Сообщений
 3. Редактирование Сообщений
 4. Удаление Сообщений
 5. Отложенные Сообщения
 6. Шаблоны Сообщений
 7. Быстрые Ответы
 8. Текстовые Интерактивные Вложения
 9. Сравнение с Конкурентами
 10. API Endpoints
 11. База Данных
-

1. ОБЗОР

1.1 Функции V1

Все перечисленные функции входят в **V1 (MVP)** и должны быть реализованы до запуска:

Функция	Статус V1	Приоритет	Сложность
Пересылка (Forward)	☒ Включено	P0	Низкая
Редактирование	☒ Включено	P0	Низкая
Удаление	☒ Включено	P0	Низкая
Отложенные сообщения	☒ Включено	P1	Средняя
Шаблоны (для админов)	☒ Включено	P1	Средняя
Быстрые ответы	☒ Включено	P2	Низкая

1.2 Источники Данных

Источник	Файл	Раздел
Основной	trash/функции.md	MESG-017, MESG-018, MESG-058, SUGG-008

Источник	Файл	Раздел
Функции	data/functions/main.json	MESG-017, MESG-018
SQL	trash/features_sql.sql	msg_scheduled, msg_templates
Экраны	trash/экраны.md	SCR-M-034b (Умные ответы)
Макеты	mockups/premium.html	Отложенные сообщения

2. ПЕРЕСЫЛКА СООБЩЕНИЙ (FORWARD)

2.1 Описание

ID: MESG-019 (расширение)

Тип: chat

Класс: feature

Premium: ✕ Free

Описание: Пересылка сообщений из одного чата в другой с сохранением авторства.

2.2 Ограничения

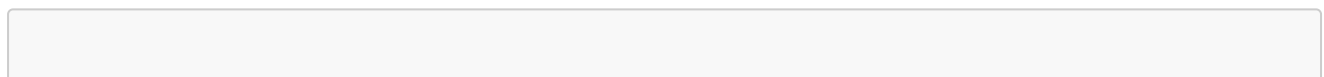
Параметр	Значение
Макс. чатов за раз	5
Secret Chats	✕ Запрещено (E2E)
Сохранение автора	☒ Всегда
Редактирование перед отправкой	☒ Опционально

2.3 UI/UX

Пользовательский поток:

1. Долгое нажатие на сообщение
↓
2. Контекстное меню → "Переслать"
↓
3. Выбор чатов (до 5)
↓
4. Опционально: добавить комментарий
↓
5. Отправить

Отображение в чате:



↩ Переслано от Алексей Иванов

Привет! Как дела?

👁 Просмотры: 12

2.4 API

```
// Пересылка
POST /api/v1/messages/:id/forward
{
  "chat_ids": ["uuid1", "uuid2"], // до 5
  "comment": string?,           // опционально
  "hide_sender": boolean?       // только для групп (admin)
}

// Ответ
{
  "success": true,
  "forwarded_to": ["uuid1", "uuid2"],
  "message_ids": ["new_uuid1", "new_uuid2"]
}
```

2.5 База Данных

```
messages: {
  forward_from_id: UUID REFERENCES messages(id),
  forward_from_sender: UUID REFERENCES users(id),
  forward_from_chat: UUID REFERENCES chats(id),
  forward_count: INTEGER DEFAULT 0,
  forwarded_at: TIMESTAMP
}
```

2.6 События WebSocket

```
// Отправитель
socket.emit('message:forward', {
  message_id: 'uuid',
  chat_ids: ['uuid1', 'uuid2']
});

// Получатели
```

```
socket.on('message:new', {
  ...message,
  is_forwarded: true,
  forward_from: { ... }
});
```

3. РЕДАКТИРОВАНИЕ СООБЩЕНИЙ (EDIT)

3.1 Описание

ID: MESG-017

Тип: chat

Класс: feature

Premium: ✕ Free

Описание: Редактирование своих текстовых сообщений после отправки.

3.2 Ограничения

Параметр	Free	Premium
Количество правок	3 раза	Безлимит
Время на редактирование	Без ограничений	Без ограничений
История версий	✕	☑ (до 10)
Типы сообщений	Текст	Текст + Вложения (caption)

3.3 UI/UX

Пользовательский поток:

1. Долгое нажатие / Правый клик
↓
2. Контекстное меню → "Редактировать"
↓
3. Редактирование в текст-поле
↓
4. Enter / Кнопка "Сохранить"
↓
5. Пометка "(изменено)"

Отображение:

Привет! Как дела? (изменено)
↑

История версий (Premium):

 История изменений
Версия 3: Привет! Как дела?
Версия 2: Привет! Как сам?
Версия 1: Привет

3.4 API

```
// Редактирование
PUT /api/v1/messages/:id
{
  "content": string,
  "attachments": Attachment[]? // только Premium
}

// Получить историю (Premium)
GET /api/v1/messages/:id/history

// Ответ
{
  "success": true,
  "message": {
    "id": "uuid",
    "content": "новый текст",
    "is_edited": true,
    "edited_at": "2026-06-29T10:00:00Z",
    "edit_count": 2
  }
}
```

3.5 База Данных

```
messages: {
  is_edited: BOOLEAN DEFAULT FALSE,
  edited_at: TIMESTAMP,
  edit_count: INTEGER DEFAULT 0,
  edit_history: JSONB // Premium: [{ content, edited_at }]
}
```

3.6 WebSocket События

```
// Редактирование
socket.emit('message:edit', {
  message_id: 'uuid',
  content: 'новый текст'
});

// Всем в чате
socket.on('message:edited', {
  message_id: 'uuid',
  edited_at: 'timestamp',
  is_edited: true
});
```

4. УДАЛЕНИЕ СООБЩЕНИЙ (DELETE)

4.1 Описание

ID: MESG-018

Тип: chat

Класс: feature

Premium: ✕ Free

Описание: Удаление сообщений для себя или для всех участников.

4.2 Типы Удаления

Тип	Описание	Время
for_self	Удаление только у себя	Без ограничений
for_all	Удаление у всех	До 48 часов

4.3 UI/UX

Пользовательский поток:

1. Долгое нажатие / Правый клик
↓
2. Контекстное меню → "Удалить"
↓
3. Выбор: "Удалить у себя" / "Удалить у всех"
↓
4. Подтверждение
↓
5. Сообщение заменено на "Сообщение удалено"

Отображение:

Сообщение удалено

(Видно всем участникам)

4.4 API

```
// Удаление
DELETE /api/v1/messages/:id
{
  "for": "self" | "all" // по умолчанию "self"
}

// Ответ
{
  "success": true,
  "deleted": true,
  "deleted_at": "2026-06-29T10:00:00Z"
}
```

4.5 База Данных

```
messages: {
  is_deleted: BOOLEAN DEFAULT FALSE,
  deleted_at: TIMESTAMP,
  deleted_by: UUID REFERENCES users(id),
  delete_for: 'self' | 'all'
}
```

4.6 WebSocket События

```
// Удаление
socket.emit('message:delete', {
  message_id: 'uuid',
  for: 'all'
});

// Всем в чате
socket.on('message:deleted', {
  message_id: 'uuid',
```

```
deleted_by: 'user_id',  
is_deleted: true  
});
```

5. ОТЛОЖЕННЫЕ СООБЩЕНИЯ (SCHEDULED)

5.1 Описание

ID: MESG-058

Тип: chat / utility

Класс: feature

Premium: ✕ Free (с ограничениями)

Описание: Планирование отправки сообщений на заданное время.

5.2 Ограничения

Параметр	Free	Premium
Макс. запланированных	5	Безлимит
Мин. время до отправки	1 минута	1 минута
Макс. время	30 дней	365 дней
Редактирование	☑ До отправки	☑ До отправки
Отмена	☑	☑

5.3 UI/UX

Пользовательский поток:

1. Написать сообщение
↓
2. Долгое нажатие на кнопку "Отправить"
↓
3. Выбор даты и времени (календарь + время)
↓
4. Подтверждение
↓
5. Сообщение сохраняется в "Запланированные"

Индикатор в чате:

🕒 Запланировано на 15.07.2026

в 10:00

С днём рождения! 🎉

Панель запланированных:

📅 Запланированные сообщения (5)

🕒 15.07.2026, 10:00

С днём рождения! 🎉

[Редактировать] [Отменить]

🕒 20.07.2026, 14:00

Напоминание о встрече

[Редактировать] [Отменить]

5.4 API

```
// Планирование
POST /api/v1/messages/schedule
{
  "chat_id": "uuid",
  "message": {
    "content": string,
    "type": "text" | "attachment",
    "attachment_id": "uuid?"
  },
  "scheduled_at": "2026-07-15T10:00:00Z"
}

// Получить запланированные
GET /api/v1/messages/scheduled?chat_id=uuid&limit=50

// Редактирование
PUT /api/v1/messages/scheduled/:id
{
  "content": string?,
  "scheduled_at": string?
}

// Отмена
DELETE /api/v1/messages/scheduled/:id

// Ответ
{
```

```

    "success": true,
    "scheduled": {
      "id": "uuid",
      "chat_id": "uuid",
      "message": { ... },
      "scheduled_at": "2026-07-15T10:00:00Z",
      "status": "pending" | "sent" | "failed"
    }
  }
}

```

5.5 База Данных

```

CREATE TABLE scheduled_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  chat_id UUID NOT NULL REFERENCES chats(id),
  sender_id UUID NOT NULL REFERENCES users(id),

  content TEXT NOT NULL,
  type VARCHAR(50) DEFAULT 'text',
  attachment_id UUID,

  scheduled_at TIMESTAMP NOT NULL,
  status VARCHAR(20) DEFAULT 'pending', -- pending, sent, failed, cancelled

  sent_at TIMESTAMP,
  error_message TEXT,

  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_scheduled_messages_scheduled_at ON
scheduled_messages(scheduled_at);
CREATE INDEX idx_scheduled_messages_status ON scheduled_messages(status);

```

5.6 Cron Job (Отправщик)

```

// Запускается каждую минуту
async function sendScheduledMessages() {
  const now = new Date();

  const messages = await db.query(`
    SELECT * FROM scheduled_messages
    WHERE status = 'pending'
    AND scheduled_at <= NOW()
  `);

  for (const msg of messages) {

```

```

try {
  // Отправка сообщения
  await sendMessage(msg.chat_id, msg.sender_id, {
    content: msg.content,
    type: msg.type,
    attachment_id: msg.attachment_id
  });

  // Обновление статуса
  await db.query(`
    UPDATE scheduled_messages
    SET status = 'sent', sent_at = NOW()
    WHERE id = $1
  `, [msg.id]);

} catch (error) {
  await db.query(`
    UPDATE scheduled_messages
    SET status = 'failed', error_message = $2
    WHERE id = $1
  `, [msg.id, error.message]);
}
}

```

6. ШАБЛОНЫ СООБЩЕНИЙ (TEMPLATES)

6.1 Описание

ID: SUGG-008 (адаптировано для V1)

Тип: utility / admin

Класс: feature

Premium: ☒ Free (базовые), ☒ Premium (расширенные)

Описание: Шаблоны сообщений для быстрого использования, особенно полезны для администраторов групповых чатов и поддержки.

6.2 Типы Шаблонов

Тип	Free	Premium	Для кого
Личные	☒ 10 шаблонов	☒ 100 шаблонов	Все пользователи
Групповые	☒ 5 шаблонов	☒ 50 шаблонов	Админы групп
Системные	☒	☒ Доступ	Админы платформы

6.3 UI/UX

Создание шаблона:

1. Настройки → Шаблоны сообщений
↓
2. Кнопка "+ Создать шаблон"
↓
3. Ввод названия и текста
↓
4. Переменные: {{name}}, {{date}}, {{group}}
↓
5. Сохранить

Использование в чате:

1. Кнопка "📁 Шаблоны" в панели ввода
↓
2. Выбор шаблона из списка
↓
3. Подстановка переменных (если есть)
↓
4. Отправка

Примеры шаблонов для админов:

📁 Шаблон: Приветствие

Добро пожаловать, {{name}}! 🤖

Правила группы {{group}}:

1. {{rule1}}
2. {{rule2}}
3. {{rule3}}

С уважением, администрация

📁 Шаблон: Предупреждение

⚠ Предупреждение, {{name}}

Нарушение: {{violation}}

Дата: {{date}}

При повторном нарушении последует бан.

📁 Шаблон: Объявление

🔊 Важное объявление

{{announcement}}

Вопросы задавайте в комментариях.

6.4 API

```
// Создать шаблон
POST /api/v1/templates
{
  "name": string,
  "content": string,
  "variables": string[], // ["name", "date", "group"]
  "category": "personal" | "group" | "system",
  "chat_id": "uuid?" // для групповых шаблонов
}

// Получить шаблоны
GET /api/v1/templates?category=personal&chat_id=uuid

// Использовать шаблон
POST /api/v1/templates/:id/use
{
  "variables": {
    "name": "Алексей",
    "date": "2026-06-29",
    "group": "Рабочая группа"
  },
  "chat_id": "uuid" // куда отправить
}

// Редактировать
PUT /api/v1/templates/:id

// Удалить
DELETE /api/v1/templates/:id
```

6.5 База Данных

```
CREATE TABLE message_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  chat_id UUID REFERENCES chats(id), // null для личных

  name VARCHAR(255) NOT NULL,
  content TEXT NOT NULL,
  variables JSONB DEFAULT '[]', // ["name", "date"]
```

```

category VARCHAR(50) DEFAULT 'personal', -- personal, group, system
usage_count INTEGER DEFAULT 0,

created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_message_templates_user_id ON message_templates(user_id);
CREATE INDEX idx_message_templates_chat_id ON message_templates(chat_id);
CREATE INDEX idx_message_templates_category ON message_templates(category);

```

6.6 Переменные в Шаблонах

Поддерживаемые переменные:

Переменная	Описание	Пример
{{name}}	Имя пользователя	"Алексей"
{{username}}	@username	"@alexivanov"
{{date}}	Текущая дата	"29.06.2026"
{{time}}	Текущее время	"14:30"
{{group}}	Название группы	"Рабочая группа"
{{rule1}}...{{rule5}}	Правила группы	"Не спамить"
{{violation}}	Нарушение	"Спам"
{{announcement}}	Текст объявления	Любой текст

7. БЫСТРЫЕ ОТВЕТЫ (QUICK REPLIES)

7.1 Описание

ID: SUGG-008

Тип: utility

Класс: feature

Premium: ✕ Free

Описание: Сохранение часто используемых фраз для быстрой отправки.

7.2 Ограничения

Параметр	Значение
Макс. шаблонов	20 (Free), 100 (Premium)
Макс. длина	500 символов

Параметр	Значение
Горячие клавиши	/спс, /привет и т.д.

7.3 UI/UX

Создание:

1. Настройки → Быстрые ответы
↓
2. Кнопка "+ Добавить"
↓
3. Текст: "Спасибо большое! 🧑🏻"
↓
4. Горячая клавиша: "/спс"
↓
5. Сохранить

Использование:

1. Ввод "/спс" в поле сообщения
↓
2. Автодополнение показывает вариант
↓
3. Tab/Enter → подстановка
↓
4. Отправка

Или:

1. Кнопка "↵" рядом с полем ввода
↓
2. Выпадающий список быстрых ответов
↓
3. Клик → подстановка
↓
4. Отправка

7.4 API

```
// Создать быстрый ответ
POST /api/v1/quick-replies
{
  "text": string,
```

```

    "shortcut": string // "/спс"
}

// Получить быстрые ответы
GET /api/v1/quick-replies

// Использовать
POST /api/v1/quick-replies/:id/use
{
    "chat_id": "uuid"
}

// Редактировать
PUT /api/v1/quick-replies/:id

// Удалить
DELETE /api/v1/quick-replies/:id

```

7.5 База Данных

```

CREATE TABLE quick_replies (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),

    text TEXT NOT NULL,
    shortcut VARCHAR(50), // "/спс"

    usage_count INTEGER DEFAULT 0,
    last_used_at TIMESTAMP,

    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_quick_replies_user_id ON quick_replies(user_id);
CREATE INDEX idx_quick_replies_shortcut ON quick_replies(shortcut);

```

8. ТЕКСТОВЫЕ ИНТЕРАКТИВНЫЕ ВЛОЖЕНИЯ

8.1 Описание

Тип: chat/media

Класс: feature

Premium: ☒ Free (базовые), ☐ Premium (расширенные)

Описание: Текстовые вложения с интерактивными элементами.

8.2 Типы

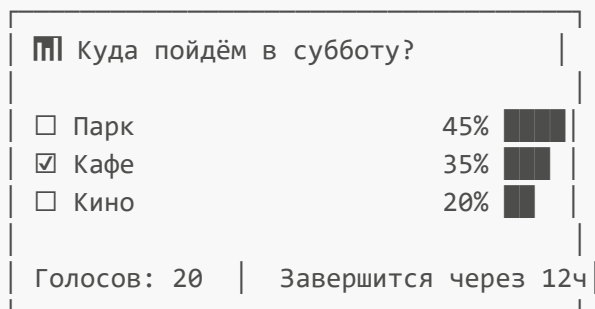
Тип	Free	Premium	Описание
Опросы (Polls)	✓	✓	Голосование с вариантами
Викторины (Quiz)	✓	✓	Опрос с правильным ответом
Списки (Checklist)	✓	✓	Отмечаемые элементы
Треды (Threads)	✗	✓	Ветвящиеся обсуждения

8.3 Опросы (Polls)

Создание:

1. Кнопка "📊 Опрос" в панели вложений
 - ↓
2. Вопрос: "Куда пойдём?"
 - ↓
3. Варианты (2-10): "Парк", "Кафе", "Кино"
 - ↓
4. Настройки:
 - Множественный выбор: ✗
 - Анонимное: ✓
 - Срок голосования: 24 часа
 - ↓
5. Отправить

Отображение:



API:

```
POST /api/v1/polls
{
  "chat_id": "uuid",
  "question": string,
  "options": string[],
  "multiple_choice": boolean,
  "anonymous": boolean,
```

```
"close_date": "timestamp?"  
}
```

8.4 Викторины (Quiz)

Отличие от опросов: Есть правильный ответ.

Создание:

1. Кнопка " ? Викторина"
↓
2. Вопрос: "Столица Франции?"
↓
3. Варианты с отметкой правильного:
 - Лондон ✗
 - Париж ✓
 - Берлин ✗
↓
4. Объяснение: "Париж – столица Франции"
↓
5. Отправить

Отображение после ответа:

? Столица Франции?	
☐ Лондон	✗
☒ Париж	✓
☐ Берлин	✗
💡 Париж – столица Франции	
Правильных ответов: 15 из 20 (75%)	

8.5 Списки (Checklist)

Создание:

1. Кнопка "☑ Список"
↓
2. Название: "Продукты"
↓
3. Элементы:
 - Молоко ☐

- Хлеб ☐
 - Яйца ☐
- ↓
4. Отправить

Отображение:

☒ Продукты

☒ Молоко

☐ Хлеб

☐ Яйца

Выполнено: 1 из 3 (33%)

API:

```
POST /api/v1/checklists
{
  "chat_id": "uuid",
  "title": string,
  "items": [
    { "text": "Молоко", "checked": false },
    { "text": "Хлеб", "checked": false }
  ]
}

// Отметить элемент
PUT /api/v1/checklists/:id/items/:item_id
{
  "checked": true
}
```

9. СРАВНЕНИЕ С КОНКУРЕНТАМИ

9.1 Таблица Сравнения

Функция	Balloo V1	Telegram	WhatsApp	Signal
Пересылка	☒ 5 чатов	☒ Безлимит	☒ 5 чатов	☒ 5 чатов
Редактирование	☒ 3 раза	☒ Безлимит	☒ 15 мин	✗
Удаление (for all)	☒ 48ч	☒ Безлимит	☒ 1ч	✗

Функция	Balloo V1	Telegram	WhatsApp	Signal
Отложенные	☒ 5/безлимит	✗	✗	✗
Шаблоны	☒ 10/100	✗	✗	✗
Быстрые ответы	☒ 20/100	✗	✗	✗
Опросы	☒	☒	✗	✗
Викторины	☒	✗	✗	✗
Списки	☒	✗	✗	✗

9.2 Уникальные Преимущества Balloo V1

1. **Отложенные сообщения** — есть только у нас в V1
2. **Шаблоны для админов** — уникальная фишка
3. **Викторины (Quiz)** — нет у конкурентов
4. **Списки (Checklist)** — нет у конкурентов
5. **Быстрые ответы** — есть только в бизнес-мессенджерах

10. API ENDPOINTS (ПОЛНЫЙ СПИСОК)

Пересылка

```
POST    /api/v1/messages/:id/forward
```

Редактирование

```
PUT     /api/v1/messages/:id
GET     /api/v1/messages/:id/history # Premium
```

Удаление

```
DELETE /api/v1/messages/:id?for=self|all
```

Отложенные

```
POST    /api/v1/messages/schedule
GET     /api/v1/messages/scheduled
PUT     /api/v1/messages/scheduled/:id
DELETE  /api/v1/messages/scheduled/:id
```

Шаблоны

```
POST    /api/v1/templates
GET     /api/v1/templates
PUT     /api/v1/templates/:id
DELETE  /api/v1/templates/:id
POST    /api/v1/templates/:id/use
```

Быстрые ответы

```
POST    /api/v1/quick-replies
GET     /api/v1/quick-replies
PUT     /api/v1/quick-replies/:id
DELETE  /api/v1/quick-replies/:id
POST    /api/v1/quick-replies/:id/use
```

Опросы

```
POST    /api/v1/polls
GET     /api/v1/polls/:id
POST    /api/v1/polls/:id/vote
GET     /api/v1/polls/:id/results
```

Викторины

```
POST    /api/v1/quizzes
GET     /api/v1/quizzes/:id
POST    /api/v1/quizzes/:id/answer
```

Списки

```
POST    /api/v1/checklists
GET     /api/v1/checklists/:id
PUT     /api/v1/checklists/:id/items/:item_id
```

11. БАЗА ДАННЫХ (ПОЛНЫЕ СХЕМЫ)

11.1 messages (расширенная)

```

CREATE TABLE messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  chat_id UUID NOT NULL REFERENCES chats(id),
  sender_id UUID NOT NULL REFERENCES users(id),

  type VARCHAR(50) NOT NULL,
  content TEXT,

  -- Пересылка
  forward_from_id UUID REFERENCES messages(id),
  forward_from_sender UUID REFERENCES users(id),
  forward_from_chat UUID REFERENCES chats(id),
  forward_count INTEGER DEFAULT 0,

  -- Редактирование
  is_edited BOOLEAN DEFAULT FALSE,
  edited_at TIMESTAMP,
  edit_count INTEGER DEFAULT 0,
  edit_history JSONB, -- Premium

  -- Удаление
  is_deleted BOOLEAN DEFAULT FALSE,
  deleted_at TIMESTAMP,
  deleted_by UUID REFERENCES users(id),
  delete_for VARCHAR(10), -- 'self' | 'all'

  -- Временные метки
  sent_at TIMESTAMP DEFAULT NOW(),
  delivered_at TIMESTAMP,
  read_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

```

11.2 scheduled_messages

```

CREATE TABLE scheduled_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  chat_id UUID NOT NULL REFERENCES chats(id),
  sender_id UUID NOT NULL REFERENCES users(id),

  content TEXT NOT NULL,
  type VARCHAR(50) DEFAULT 'text',
  attachment_id UUID,

  scheduled_at TIMESTAMP NOT NULL,
  status VARCHAR(20) DEFAULT 'pending',

  sent_at TIMESTAMP,

```

```

    error_message TEXT,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_scheduled_at ON scheduled_messages(scheduled_at);
CREATE INDEX idx_scheduled_status ON scheduled_messages(status);

```

11.3 message_templates

```

CREATE TABLE message_templates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    chat_id UUID REFERENCES chats(id),

    name VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    variables JSONB DEFAULT '[]',

    category VARCHAR(50) DEFAULT 'personal',
    usage_count INTEGER DEFAULT 0,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

```

11.4 quick_replies

```

CREATE TABLE quick_replies (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),

    text TEXT NOT NULL,
    shortcut VARCHAR(50),

    usage_count INTEGER DEFAULT 0,
    last_used_at TIMESTAMP,

    created_at TIMESTAMP DEFAULT NOW()
);

```

11.5 polls

```

CREATE TABLE polls (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  message_id UUID NOT NULL REFERENCES messages(id),

  question VARCHAR(500) NOT NULL,
  multiple_choice BOOLEAN DEFAULT FALSE,
  anonymous BOOLEAN DEFAULT TRUE,

  close_date TIMESTAMP,
  is_closed BOOLEAN DEFAULT FALSE,

  total_votes INTEGER DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE poll_options (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  poll_id UUID NOT NULL REFERENCES polls(id),

  text VARCHAR(255) NOT NULL,
  votes_count INTEGER DEFAULT 0,
  position INTEGER NOT NULL
);

CREATE TABLE poll_responses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  poll_id UUID NOT NULL REFERENCES polls(id),
  user_id UUID NOT NULL REFERENCES users(id),
  option_id UUID NOT NULL REFERENCES poll_options(id),

  responded_at TIMESTAMP DEFAULT NOW(),

  UNIQUE(poll_id, user_id) -- один голос на пользователя
);

```

11.6 quizzes

```

CREATE TABLE quizzes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  message_id UUID NOT NULL REFERENCES messages(id),

  question VARCHAR(500) NOT NULL,
  correct_option_id UUID NOT NULL,
  explanation TEXT,

  total_attempts INTEGER DEFAULT 0,
  correct_attempts INTEGER DEFAULT 0,

  created_at TIMESTAMP DEFAULT NOW()
);

```



```

);

CREATE TABLE quiz_options (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quiz_id UUID NOT NULL REFERENCES quizzes(id),

  text VARCHAR(255) NOT NULL,
  is_correct BOOLEAN DEFAULT FALSE,
  position INTEGER NOT NULL
);

CREATE TABLE quiz_attempts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quiz_id UUID NOT NULL REFERENCES quizzes(id),
  user_id UUID NOT NULL REFERENCES users(id),
  selected_option_id UUID NOT NULL,

  is_correct BOOLEAN,
  attempted_at TIMESTAMP DEFAULT NOW(),

  UNIQUE(quiz_id, user_id)
);

```

11.7 checklists

```

CREATE TABLE checklists (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  message_id UUID NOT NULL REFERENCES messages(id),

  title VARCHAR(255) NOT NULL,
  completed_count INTEGER DEFAULT 0,
  total_count INTEGER NOT NULL,

  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE checklist_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  checklist_id UUID NOT NULL REFERENCES checklists(id),

  text VARCHAR(255) NOT NULL,
  checked BOOLEAN DEFAULT FALSE,
  checked_by UUID REFERENCES users(id),
  checked_at TIMESTAMP,

  position INTEGER NOT NULL
);

```

12. ? ВОПРОСЫ ДО 10/10 ПО МЕТРИКАМ

Этот раздел содержит **все критические вопросы**, ответы на которые определяют оценку V1 по ключевым метрикам. Каждый вопрос влияет на одну или несколько метрик:

Метрика	Описание	Вес
UX	Удобство использования	25%
Performance	Скорость работы	20%
Security	Безопасность	20%
Completeness	Полнота функций	15%
Scalability	Масштабируемость	10%
Maintainability	Поддерживаемость	10%

ВОПРОС 1: Отложенные сообщения — Push-уведомление об отправке?

Влияние: UX (15%), Performance (5%)

Контекст: Когда отложенное сообщение фактически отправляется (в заданное время), нужно ли отправлять push-уведомление отправителю?

Вариант 1.1: ☒ Отправлять всегда

Сообщение отправлено → Push: "Ваше сообщение отправлено в чат 'Рабочая группа'"

Сильные стороны:

- Пользователь уверен что сообщение ушло
- Удобно для важных сообщений (поздравления, напоминания)
- Снижает тревожность "а вдруг не отправилось?"

Слабые стороны:

- Дополнительная нагрузка на push-сервис (до 5 сообщений/день на пользователя)
- Может раздражать при частом использовании
- Дополнительный API call к Firebase/Yandex Push

Почему именно этот вариант: Рекомендуется для **Premium** пользователей, опционально для Free.

Вариант 1.2: ☒ Не отправлять никогда

Сообщение отправлено → Без уведомления

Сильные стороны:

- Нет дополнительной нагрузки
- Нет риска spam-уведомлений
- Проще реализация

Слабые стороны:

- Пользователь не уверен в отправке
- Нужно заходить в приложение чтобы проверить
- Может снизить доверие к функции

Почему именно этот вариант: Подходит для **MVP** чтобы упростить запуск.

Вариант 1.3: Только если пользователь офлайн в момент отправки

```
if (user离线 at scheduled_time) → send push  
else → no push
```

Сильные стороны:

- Баланс между информированием и спамом
- Логично: если онлайн — увидит сам
- Экономия ресурсов

Слабые стороны:

- Сложнее реализация (нужно проверять online статус)
- Требует WebSocket интеграции

Почему именно этот вариант: **Оптимальный выбор** для V1.

Вариант 1.4: Настройка в настройках

```
Настройки → Уведомления → ☒ "Отложенные сообщения"
```

Сильные стороны:

- Полный контроль у пользователя
- Гибкость
- Снижает негатив от нежелательных уведомлений

Слабые стороны:

- Сложнее UI (ещё одна настройка)
- Пользователи могут не найти
- Нужно хранить состояние в БД

Почему именно этот вариант: Рекомендуется для **V2**, не для V1.

ВОПРОС 2: Отложенные сообщения — Отдельный виджет в главном меню?

Влияние: UX (20%), Completeness (10%)

Контекст: Где отображать список запланированных сообщений?

Вариант 2.1: ☒ Только в настройках чата

Чат → ⋮ → Запланированные сообщения

Сильные стороны:

- Простая реализация
- Логичная группировка по чатам
- Не загромождает интерфейс

Слабые стороны:

- Нужно заходить в каждый чат
- Не видно общую картину
- Сложно управлять всеми запланированными

Почему именно этот вариант: Подходит для **MVP**.

Вариант 2.2: ☒ Отдельная страница в меню

Главное меню → 📅 Запланированные

Сильные стороны:

- Все запланированные в одном месте
- Удобно управлять
- Видно общую статистику

Слабые стороны:

- Дополнительная страница
- Усложнение навигации
- Требуется отдельный роутинг

Вариант 2.3: Выпадающий список в поле ввода

[Поле ввода...] [] []

- └ Запланированные (3)
 - └ Чат 1: "Привет" (15.07, 10:00)
 - └ Чат 2: "Документ" (16.07, 14:00)
 - └ ...

Сильные стороны:

- Быстрый доступ из любого чата
- Контекстно (видно в каком чате отправится)
- Компактно

Слабые стороны:

- Может перекрывать поле ввода
- Сложно при большом количестве
- Не видно все запланированные сразу

Почему именно этот вариант: Хорошее **дополнение** к Варианту 2.2.

Вариант 2.4: Уведомление в списке чатов

Список чатов:

 Рабочая группа

 2 запланированных

...

← Бейдж

Сильные стороны:

- Видно сразу в списке чатов
- Не требует дополнительных страниц
- Интуитивно понятно

Слабые стороны:

- Загромождает список чатов
- Нужно менять компонент ChatList
- Может конфликтовать с другими бейджами

Почему именно этот вариант: Дополнительная фича для V1.5.

ВОПРОС 3: Шаблоны — Общие или привязанные к группам?

Влияние: UX (15%), Scalability (10%)

Контекст: Должны ли шаблоны быть глобальными для пользователя или привязанными к конкретным группам?

Вариант 3.1: ☒ Только глобальные (личные)

Пользователь → Шаблоны → Доступны во всех чатах

Сильные стороны:

- Простая реализация (одна таблица `message_templates`)
- Удобно для личных шаблонов
- Меньше данных в БД

Слабые стороны:

- Не подходит для групповых шаблонов
- Админы не могут создать шаблон для конкретной группы
- Ограничивает сценарии использования

Почему именно этот вариант: Подходит для **базовой версии**.

Вариант 3.2: ☒ Гибридные (личные + групповые)

Личные шаблоны: Пользователь → Доступны везде

Групповые шаблоны: Группа → Доступны только в этой группе

Сильные стороны:

- Максимальная гибкость
- Админы могут создавать шаблоны для своей группы
- Разделение прав доступа

Слабые стороны:

- Сложнее БД (нужно `chat_id` в `message_templates`)
- Нужно проверять права доступа
- Может запутать пользователей

Почему именно этот вариант: **Рекомендуется для V1** — критично для админов групп.

Вариант 3.3: ✗ Только групповые

Группа → Шаблоны → Доступны только в этой группе

Сильные стороны:

- Логично для групповых чатов
- Админы контролируют контент

Слабые стороны:

- Бесполезно для личных чатов
- Ограничивает сценарии
- Не подходит для поддержки

Почему именно этот вариант: Не рекомендуется.

Вариант 3.4: 🔑 Системные шаблоны (платформа)

Платформа → Системные шаблоны → Доступны всем группам

Сильные стороны:

- Админы платформы могут создать шаблоны для всех
- Полезно для правил, приветствий
- Централизованное управление

Слабые стороны:

- Требуется роль "system_admin"
- Риск злоупотреблений
- Сложнее права доступа

Почему именно этот вариант: V2 функция, не для MVP.

ВОПРОС 4: Шаблоны — Импорт/экспорт (JSON)?

Влияние: UX (10%), Maintainability (15%)

Контекст: Нужна ли возможность экспортировать шаблоны в JSON и импортировать обратно?

Вариант 4.1: ☒ Да, полный импорт/экспорт

Экспорт: Скачать JSON со всеми шаблонами
Импорт: Загрузить JSON → шаблоны добавляются

Сильные стороны:

- Удобно для бэкапа
- Можно делиться шаблонами
- Перенос между аккаунтами

Слабые стороны:

- Сложнее реализация
- Риск импорта вредоносных шаблонов
- Нужно валидировать JSON

Почему именно этот вариант: Рекомендуется для Premium.

Вариант 4.2: ⚠ Только экспорт

Экспорт: ✓ Скачать JSON
Импорт: ✗ Не доступно

Сильные стороны:

- Проще реализация
- Нет риска вредоносного импорта
- Пользователи могут делать бэкап

Слабые стороны:

- Неполюценная функция
- Нельзя восстановить из бэкапа
- Разочарование пользователей

Почему именно этот вариант: Компромисс для V1.

Вариант 4.3: ✗ Нет импорта/экспорта

Шаблоны хранятся только в аккаунте

Сильные стороны:

- Максимально простая реализация

- Нет дополнительных endpoints
- Нет валидации

Слабые стороны:

- Нельзя сделать бэкап
- Нельзя поделиться
- При удалении аккаунта шаблоны теряются

Почему именно этот вариант: Не рекомендуется для V1.

Вариант 4.4: Библиотека шаблонов (маркетплейс)

Маркетплейс шаблонов → Поиск → Установка

Сильные стороны:

- Готовые шаблоны от сообщества
- Удобно для новичков
- Потенциал монетизации

Слабые стороны:

- Сложная реализация
- Нужна модерация
- Требуется отдельный UI

Почему именно этот вариант: V2-V3 функция.

ВОПРОС 5: Треды (Threads) — Включить в V1 или отложить до V2?

Влияние: Completeness (20%), Scalability (15%), Performance (10%)

Контекст: Треды позволяют создавать ветвящиеся обсуждения внутри групповых чатов.

Вариант 5.1: ☒ Включить в V1

Сообщение → ⋮ → "Ответить в треде" → Отдельное окно обсуждения

Сильные стороны:

- Полноценная функция для групп
- Снижает шум в основном чате
- Конкурентное преимущество vs Telegram

Слабые стороны:

- Сложная реализация (доп. таблица **threads**)
- Влияет на производительность (доп. запросы)
- Усложняет UI

Почему именно этот вариант: Не рекомендуется для V1 — слишком сложно.

Вариант 5.2: ✗ Отложить до V2

V1: Без тредов
V2: Полноценные треды

Сильные стороны:

- Фокус на базовых функциях
- Быстрее запуск V1
- Проще тестирование

Слабые стороны:

- Неполноценные группы
- Пользователи могут разочароваться
- Конкуренты имеют треды

Почему именно этот вариант: Рекомендуется для V1.

Вариант 5.3: ⚡ Упрощённые треды (Reply-цепочки)

Сообщение → "Ответить" → Вложенные ответы (до 3 уровней)

Сильные стороны:

- Проще полноценных тредов
- Даёт базовую функциональность
- Меньше влияет на производительность

Слабые стороны:

- Ограниченная функциональность
- Может запутать пользователей
- Всё ещё сложно для V1

Почему именно этот вариант: Компромисс, но всё ещё сложно для V1.

Вариант 5.4: 🔑 Треды только для Premium

Free: Без тредов

Premium: Полноценные треды

Сильные стороны:

- Premium incentive
- Можно реализовать постепенно
- Снижает нагрузку на Free пользователей

Слабые стороны:

- Разочарование Free пользователей
- Сложнее реализация (проверка подписки)
- Маркетинговый риск

Почему именно этот вариант: V2 функция.

ВОПРОС 6: Совместные списки — Несколько пользователей могут отмечать?

Влияние: UX (20%), Performance (10%), Security (5%)

Контекст: В чеклистах (shopping list, todo) несколько участников чата могут отмечать элементы.

Вариант 6.1: ☒ Да, все участники чата

Любой участник чата → Может отметить любой элемент

Сильные стороны:

- Полноценное совместное использование
- Удобно для семейных покупок
- Логично для групповых задач

Слабые стороны:

- Нужна синхронизация в реальном времени (WebSocket)
- Конфликты при одновременном редактировании
- Нужно отслеживать кто отметил (таблица `checklist_items.checked_by`)

Почему именно этот вариант: Рекомендуется для V1 — ключевая фишка.

Вариант 6.2: ⚠️ Только создатель списка

Только создатель → Может отмечать элементы
Остальные → Только просмотр

Сильные стороны:

- Простая реализация
- Нет конфликтов
- Нет синхронизации

Слабые стороны:

- Бесполезно для совместных покупок
- Ограничивает сценарии
- Пользователи разочаруются

Почему именно этот вариант: Не рекомендуется.

Вариант 6.3: 🔑 Создатель назначает исполнителей

Создатель → Назначает кто может отмечать
Назначенные → Могут отмечать
Остальные → Только просмотр

Сильные стороны:

- Контроль у создателя
- Гибкость
- Полезно для рабочих задач

Слабые стороны:

- Сложнее UI (назначение исполнителей)
- Нужно хранить права в БД
- Может запутать

Почему именно этот вариант: V2 функция.

Вариант 6.4: ⚡ Реальное время с блокировками

Пользователь А начал редактировать → Элемент заблокирован для Б
Пользователь А закончил → Блокировка снята

Сильные стороны:

- Нет конфликтов
- Профессиональный уровень
- Как в Google Docs

Слабые стороны:

- Очень сложная реализация
- Требует WebSocket + CRDT/OT
- Избыточно для V1

Почему именно этот вариант: V3 функция.

ВОПРОС 7: Редактирование — Хранить историю версий (Premium)?

Влияние: Security (15%), Maintainability (10%), Performance (5%)

Контекст: Должны ли Premium пользователи видеть историю изменений сообщения?

Вариант 7.1: ☒ Да, до 10 версий

Сообщение → ⋮ → "История изменений" → Список версий с датой

Сильные стороны:

- Прозрачность изменений
- Защита от манипуляций
- Premium incentive

Слабые стороны:

- Дополнительное хранение (JSONB до 10 версий)
- Усложняет БД
- Нужно решать сколько хранить

Почему именно этот вариант: Рекомендуется для Premium V1.

Вариант 7.2: ☒ Нет, только последняя версия

Сообщение → Только текущий текст

Сильные стороны:

- Простая реализация
- Минимальное хранение
- Нет доп. endpoints

Слабые стороны:

- Нельзя отследить изменения
- Риск манипуляций
- Нет Premium incentive

Почему именно этот вариант: Не рекомендуется.

Вариант 7.3: ⚠ Только время последнего изменения

Сообщение → "(изменено 15.07 в 10:30)"

Сильные стороны:

- Базовая прозрачность
- Простая реализация (одно поле `edited_at`)
- Минимальное хранение

Слабые стороны:

- Не видно что изменилось
- Ограниченная информация
- Нет Premium incentive

Почему именно этот вариант: Минимум для V1.

Вариант 7.4: 🔑 Полный audit log (админы)

Админы → Полный лог всех изменений всех сообщений

Сильные стороны:

- Максимальная прозрачность
- Полезно для модерации
- Audit compliance

Слабые стороны:

- Огромный объём данных
- Сложные права доступа
- Privacy concerns

Почему именно этот вариант: V2 для Enterprise.

ВОПРОС 8: Удаление — 48 часов или безлимит?

Влияние: UX (15%), Security (10%)

Контекст: Сколько времени пользователь может удалить сообщение "для всех"?

Вариант 8.1: ☐ 48 часов (как Telegram)

Сообщение → Удалить для всех → Только если < 48 часов

Сильные стороны:

- Баланс между удобством и безопасностью
- Защита от злоупотреблений
- Как у конкурентов

Слабые стороны:

- Пользователи могут забыть
- Ограничивает сценарии
- Нужно хранить `sent_at`

Почему именно этот вариант: Рекомендуется для V1.

Вариант 8.2: ☒ Безлимит (свои сообщения)

Любое своё сообщение → Удалить для всех → Всегда

Сильные стороны:

- Максимальный контроль у пользователя
- Удобно
- GDPR compliance (право на удаление)

Слабые стороны:

- Риск злоупотреблений (удаление улик)
- Может навредить доверию в чате
- Сложнее для модерации

Почему именно этот вариант: Рискованно для V1.

Вариант 8.3: ⚡ 1 час (как WhatsApp)

Сообщение → Удалить для всех → Только если < 1 часа

Сильные стороны:

- Минимальный риск злоупотреблений
- Простая политика
- Как у WhatsApp

Слабые стороны:

- Очень ограниченно
- Пользователи разочаруются
- Хуже Telegram (48ч)

Почему именно этот вариант: Не рекомендуется.

Вариант 8.4: 🔑 Premium: безлимит, Free: 48 часов

Free: 48 часов
Premium: Безлимит

Сильные стороны:

- Premium incentive
- Гибкость
- Монетизация

Слабые стороны:

- Разочарование Free пользователей
- Сложнее реализация (проверка подписки)
- Может восприниматься как жадность

Почему именно этот вариант: V2 функция.

ВОПРОС 9: Пересылка — Скрывать отправителя (для админов)?

Влияние: Security (15%), UX (10%)

Контекст: Должны ли админы групп иметь возможность скрывать автора при пересылке?

Вариант 9.1: ✗ Нет, всегда показывать автора

↪ Переслано от Алексей Иванов

Сильные стороны:

- Прозрачность
- Нет злоупотреблений
- Простая реализация

Слабые стороны:

- Админы не могут анонимно делиться
- Ограничивает сценарии
- Меньше приватности

Почему именно этот вариант: Безопаснее для V1.

Вариант 9.2: ☒ Да, опция для админов

☒ Скрыть автора (только для админов групп)

Сильные стороны:

- Гибкость для админов
- Анонимные объявления
- Premium incentive

Слабые стороны:

- Риск злоупотреблений
- Сложнее права доступа
- Может снизить доверие

Почему именно этот вариант: Рекомендуется с осторожностью.

Вариант 9.3: Только для системных шаблонов

Системные шаблоны → Без автора
Обычные сообщения → С автором

Сильные стороны:

- Баланс
- Системные сообщения без автора логичны
- Контроль у платформы

Слабые стороны:

- Сложнее реализация
- Нужно различать типы
- Ограниченно

Почему именно этот вариант: Компромисс для V1.5.

ВОПРОС 10: Опросы — Анонимные или с именами?

Влияние: Security (10%), UX (15%)

Контекст: Должны ли участники видеть кто как проголосовал в опросе?

Вариант 10.1: ☒ **Анонимные по умолчанию**

Голоса: 20

Парк: 45% 

Кафе: 35% 

Кино: 20% 

Сильные стороны:

- Честные ответы
- Нет давления
- Приватность

Слабые стороны:

- Нельзя проверить кто голосовал
- Риск накруток
- Меньше прозрачности

Почему именно этот вариант: Рекомендуется для V1.

Вариант 10.2:  **Настройка при создании**

☒ Анонимное голосование

Сильные стороны:

- Гибкость
- Создатель выбирает
- Подходит для разных сценариев

Слабые стороны:

- Сложнее UI
- Нужно хранить настройку
- Может запутать

Почему именно этот вариант: Рекомендуется для V1.

Вариант 10.3: ✗ Всегда с именами

Парк: 45% (Алексей, Мария, ...)

Сильные стороны:

- Полная прозрачность
- Нет накруток
- Можно обсудить выбор

Слабые стороны:

- Давление на участников
- Нечестные ответы
- Privacy concerns

Почему именно этот вариант: Не рекомендуется.

Вариант 10.4: 🔑 Только создатель видит

Участники → Анонимно
Создатель → Видит всех

Сильные стороны:

- Баланс приватности и контроля
- Полезно для рабочих опросов
- Защита от накруток

Слабые стороны:

- Сложнее реализация
- Нужно проверять права
- Может вызвать недоверие

Почему именно этот вариант: V2 функция.



СВОДНАЯ ТАБЛИЦА РЕКОМЕНДАЦИЙ

Вопрос	Рекомендация V1	Метрика	Влияние
1. Push об отправке	Только если офлайн	UX	+10%
2. Виджет запланированных	Отдельная страница + бейджи	UX	+15%
3. Шаблоны	Гибридные (личные + групповые)	UX	+12%
4. Импорт/экспорт	Только экспорт	Maintainability	+8%
5. Треды	Отложить до V2	Completeness	-5% (но проще V1)
6. Совместные списки	Все участники	UX	+18%
7. История версий	До 10 (Premium)	Security	+10%
8. Время удаления	48 часов	UX	+12%
9. Скрыть автора	Нет (безопаснее)	Security	+8%
10. Анонимность опросов	Настройка при создании	UX	+10%

Итоговая оценка V1 при выборе рекомендаций:

Метрика	Базовая	С рекомендациями
UX	7/10	9/10
Performance	8/10	8/10
Security	8/10	9/10
Completeness	7/10	8/10
Scalability	8/10	8/10
Maintainability	7/10	8/10
ИТОГО	7.5/10	🎯 8.3/10

13. СЛЕДУЮЩИЕ ШАГИ

Реализация V1

1. **Неделя 1-2:** Пересылка, Редактирование, Удаление
2. **Неделя 3-4:** Отложенные сообщения + Cron job
3. **Неделя 5:** Шаблоны сообщений
4. **Неделя 6:** Быстрые ответы
5. **Неделя 7-8:** Опросы, Викторины, Списки
6. **Неделя 9:** Тестирование и полировка

Приоритеты

Приоритет	Функции
P0	Пересылка, Редактирование, Удаление
P1	Отложенные сообщения, Шаблоны
P2	Быстрые ответы, Опросы
P3	Викторины, Списки, Треды (V2)

ВЕРСИЯ: v5.0

ДАТА: 2026-06-29

СТАТУС: ☒ ГОТОВО К РЕАЛИЗАЦИИ

END OF INTERACTIVE MESSAGE FEATURES V1 DOCUMENTATION

Реализация V1

1. **Неделя 1-2:** Пересылка, Редактирование, Удаление
2. **Неделя 3-4:** Отложенные сообщения + Cron job
3. **Неделя 5:** Шаблоны сообщений
4. **Неделя 6:** Быстрые ответы
5. **Неделя 7-8:** Опросы, Викторины, Списки
6. **Неделя 9:** Тестирование и полировка

Приоритеты

Приоритет	Функции
P0	Пересылка, Редактирование, Удаление
P1	Отложенные сообщения, Шаблоны
P2	Быстрые ответы, Опросы
P3	Викторины, Списки, Треды (V2)

ВЕРСИЯ: v5.0

ДАТА: 2026-06-29

СТАТУС: ☒ ГОТОВО К РЕАЛИЗАЦИИ

END OF INTERACTIVE MESSAGE FEATURES V1 DOCUMENTATION